Setting up an ML Workflows: DVC, Google Drive & GitHub Actions

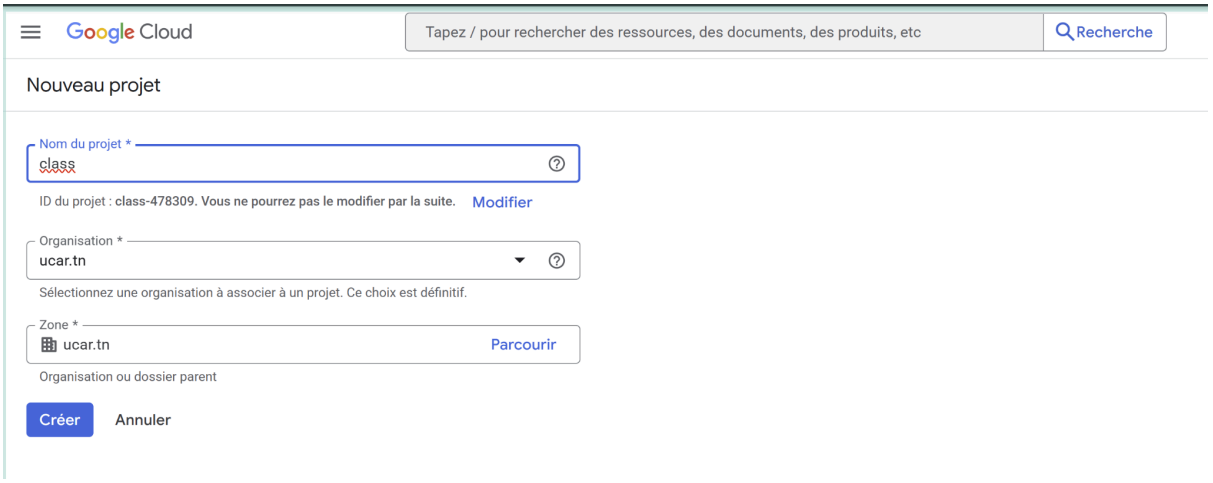
14/11/2025

1. Introduction

This document provides a comprehensive guide for setting up a robust machine learning workflow that integrates DVC for data versioning, Google Drive for scalable storage, and GitHub Actions for automated CI/CD pipelines, addressing critical challenges in ML projects such as managing large datasets beyond Git's capabilities, ensuring experiment reproducibility across environments, and automating training processes while leveraging Google Drive's cost-effective storage through service account authentication and shared folder configurations for seamless team collaboration in production-ready ML environments.

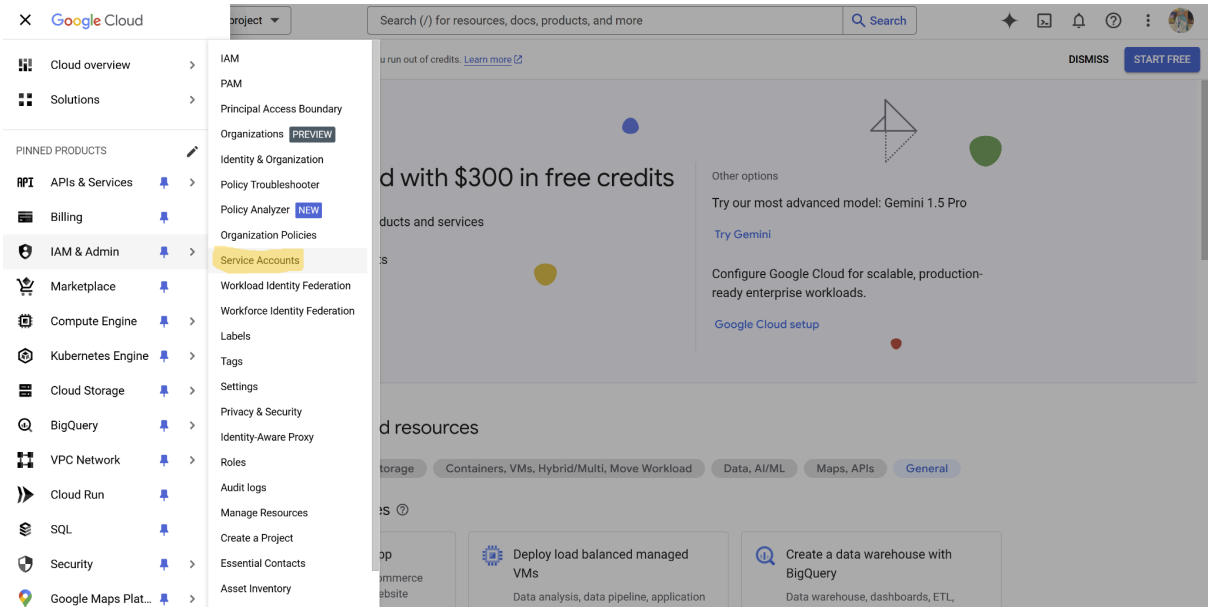
2. Steps

Create a project

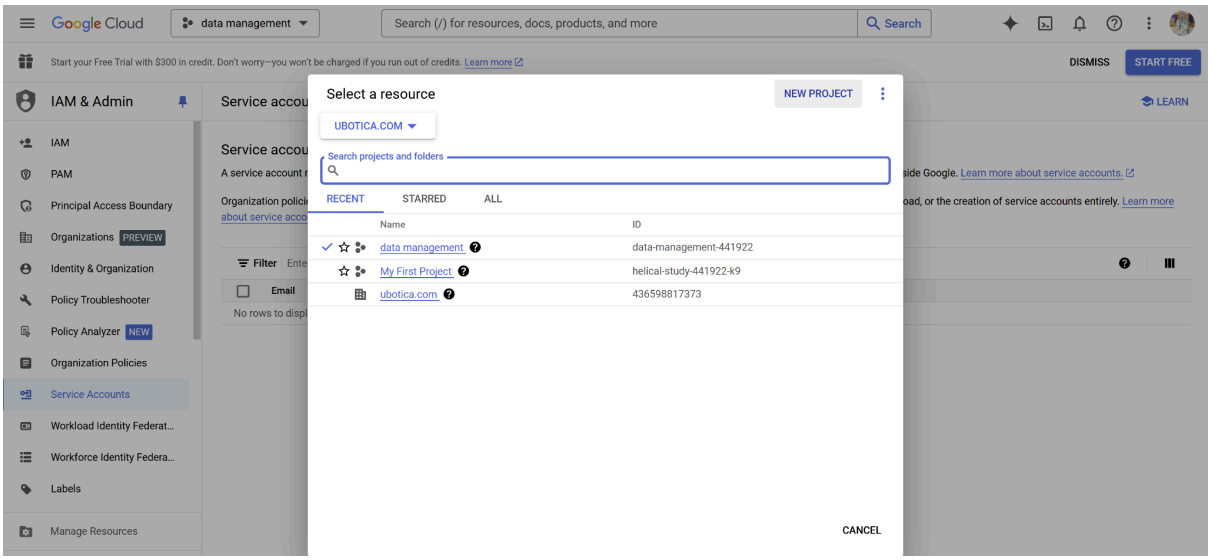


2.1 Setup up Google Service

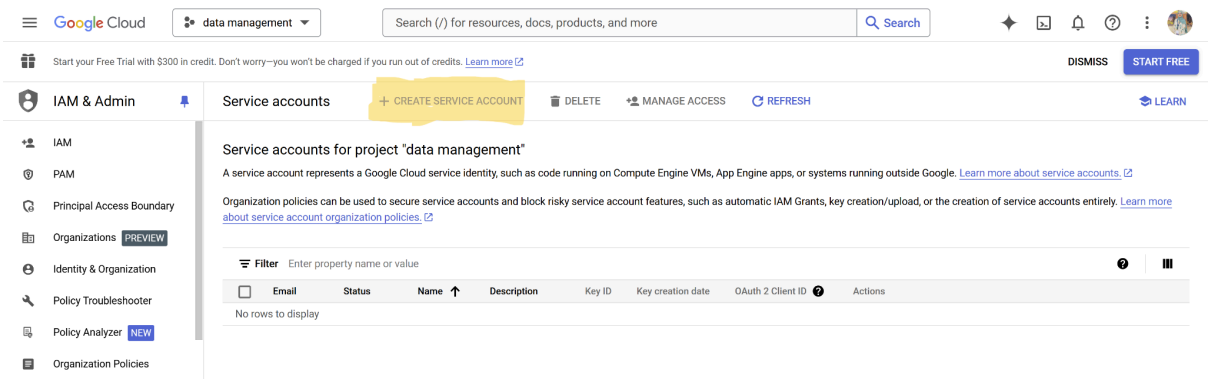
1. Access https://console.cloud.google.com/
2. Create a service account, by going to **IAM & Admin** in the left-hand menu selection and selecting **Service Accounts**.



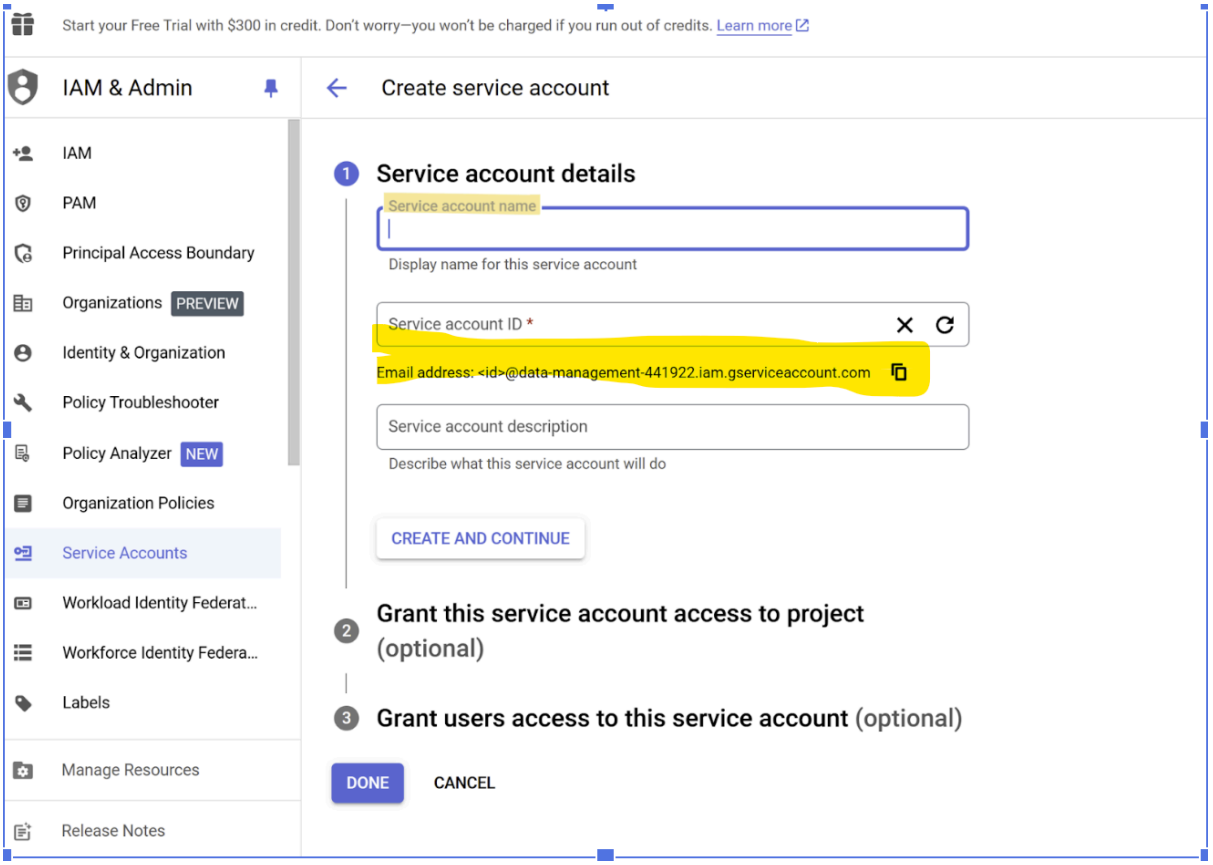
3. You should first create a project (e.g datamanagement), and then select it



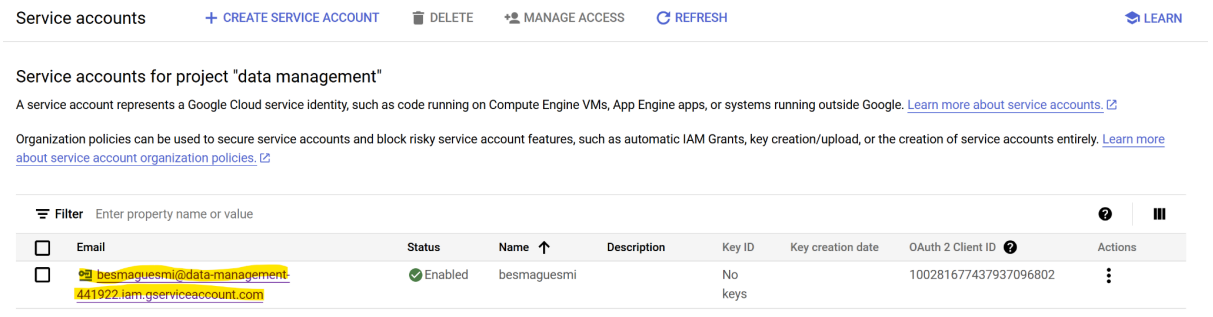
4. Once the project is created and selected, you should be able to see a tab called “CREATE SERVICE ACCOUNT”, as shown below, click it!

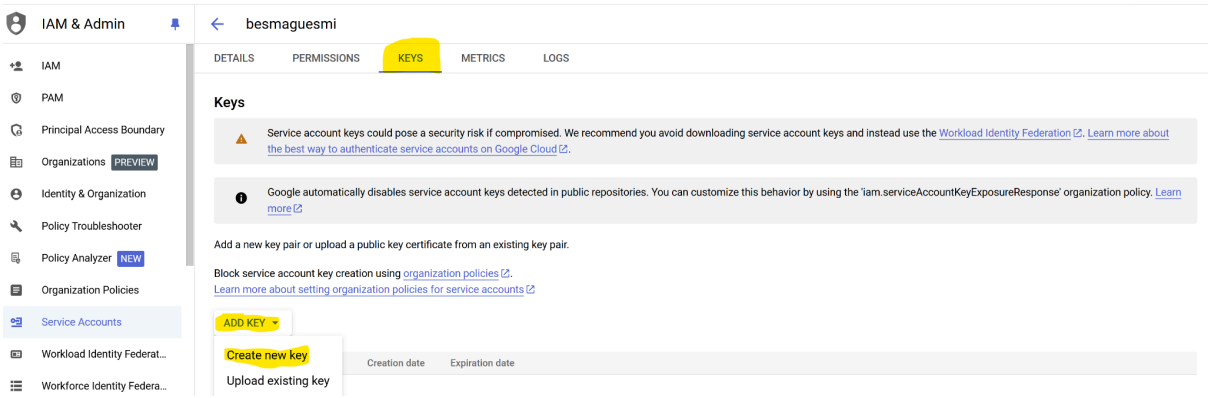


5. Enter your service account name (e.g. namesurname), click “CREATE AND CONTINUE” and then you should add all user accounts to which you need to grant access., and then click “DONE”. *Also you need to save the email address in your notebook since you will be using it later to give access to your shared drive folder.*

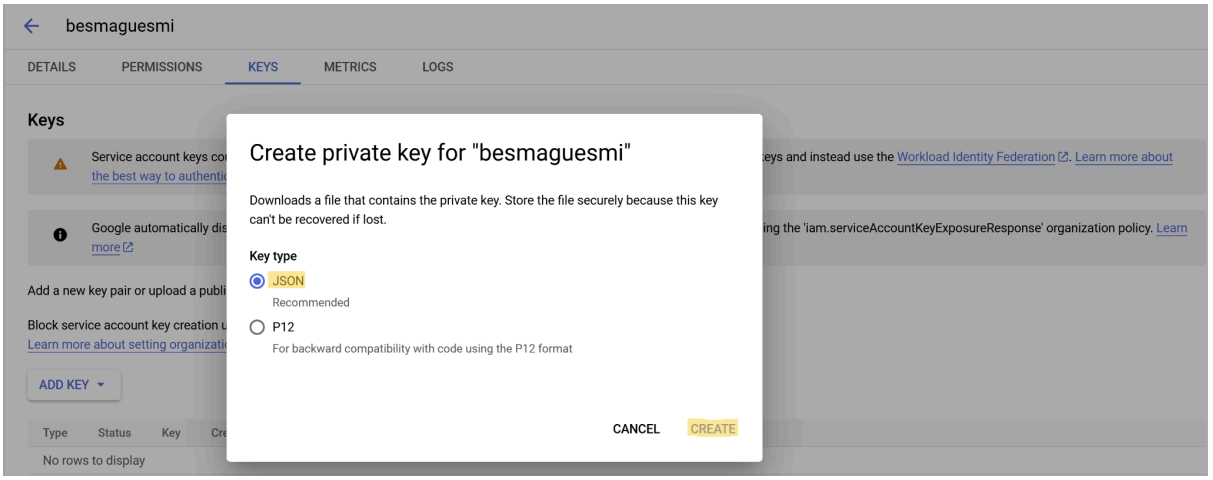


6. Now you can see your service account; click on it and go to the **“KEYS”** tab, after that, press “**ADD KEY**” bottom and create your new key.

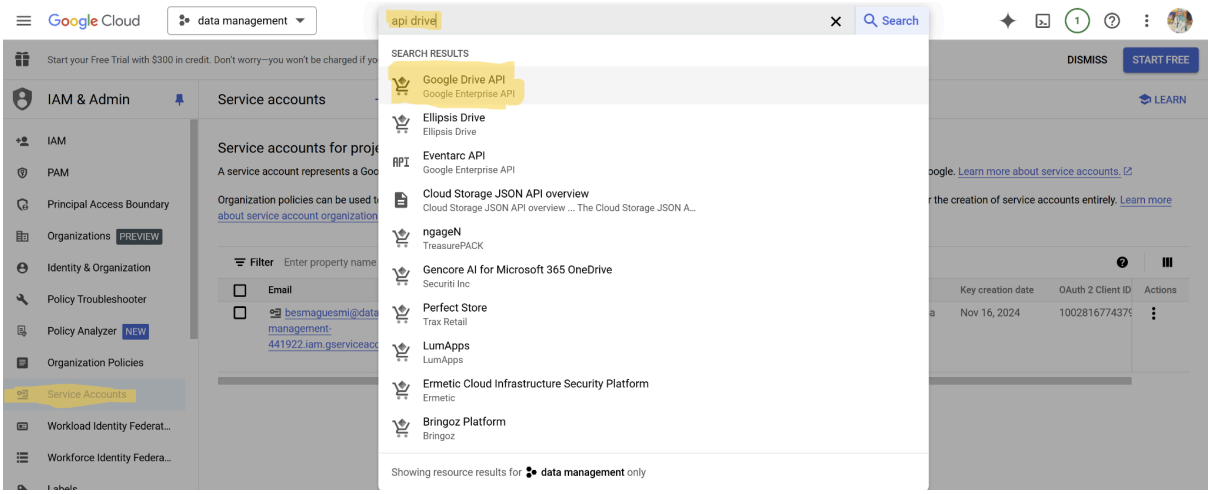


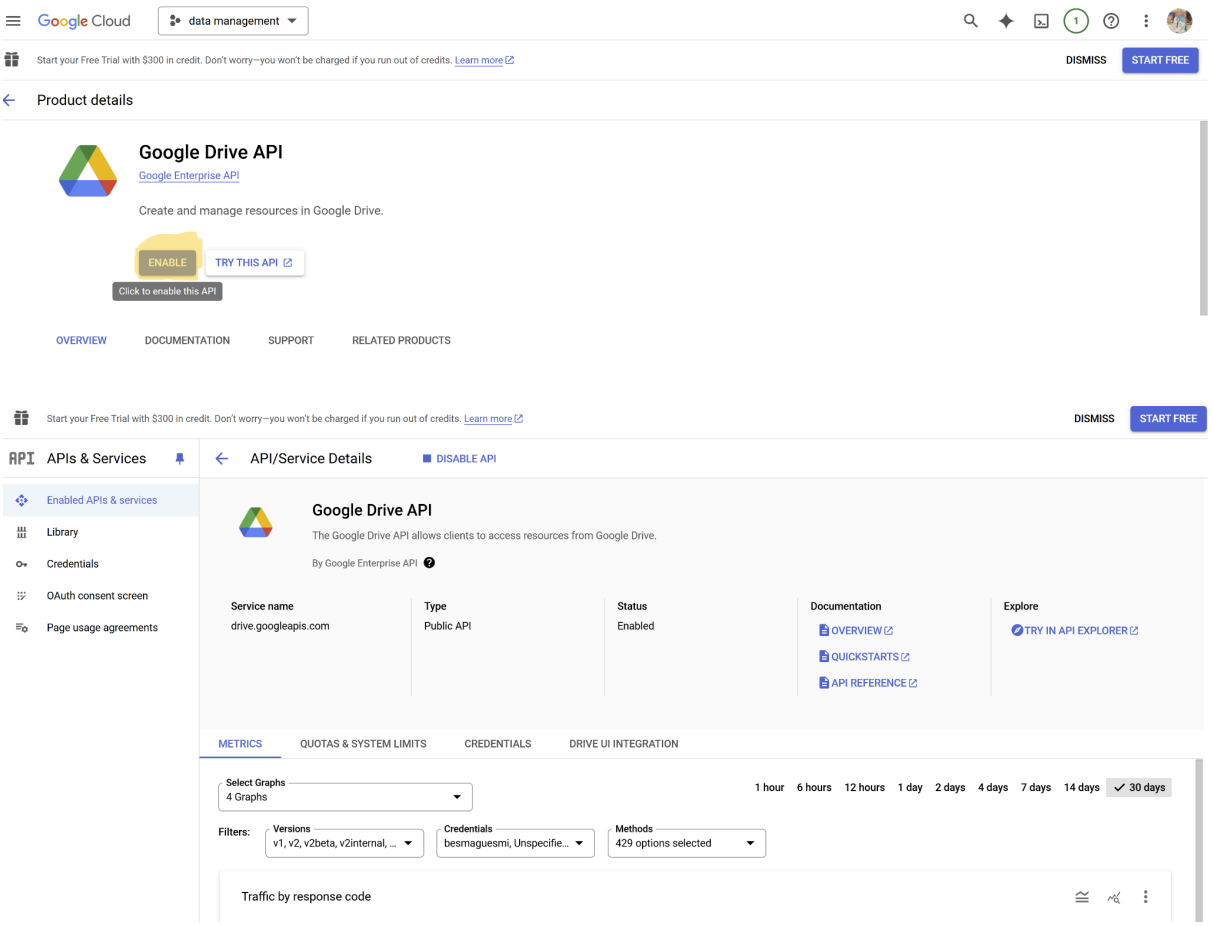


7. Once you click “Create new key”, choose “JSON”, and click CREATE. You will get your json credentials saved automatically. You should store the API key under your project folder and do not commit it to GitHub.



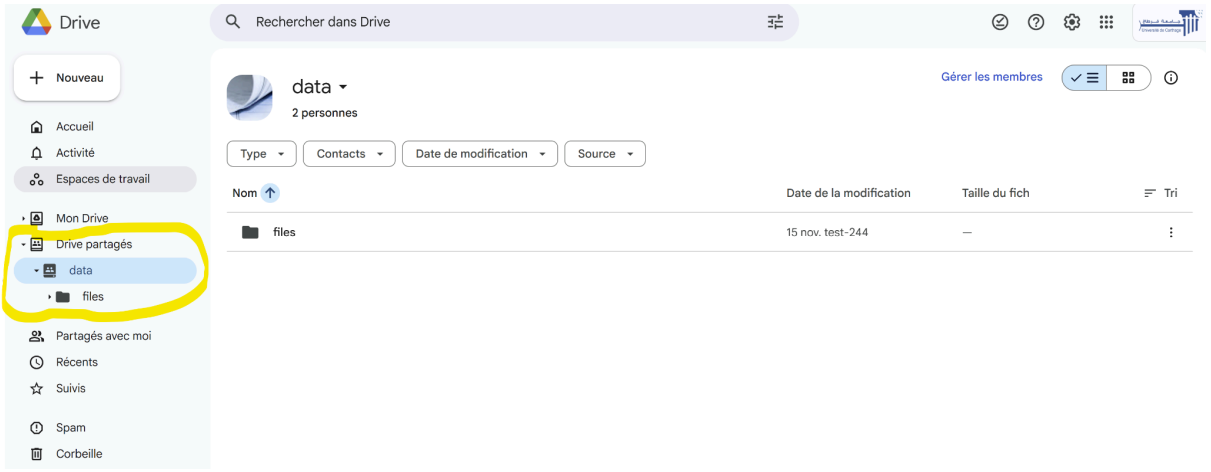
8. Additionally, we should enable Google Drive API to be able to allow the interaction of DVC with Google Drive. Enable it by clicking on the **“ENABLE”** bottom.



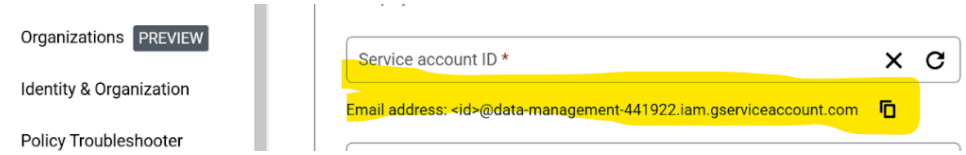


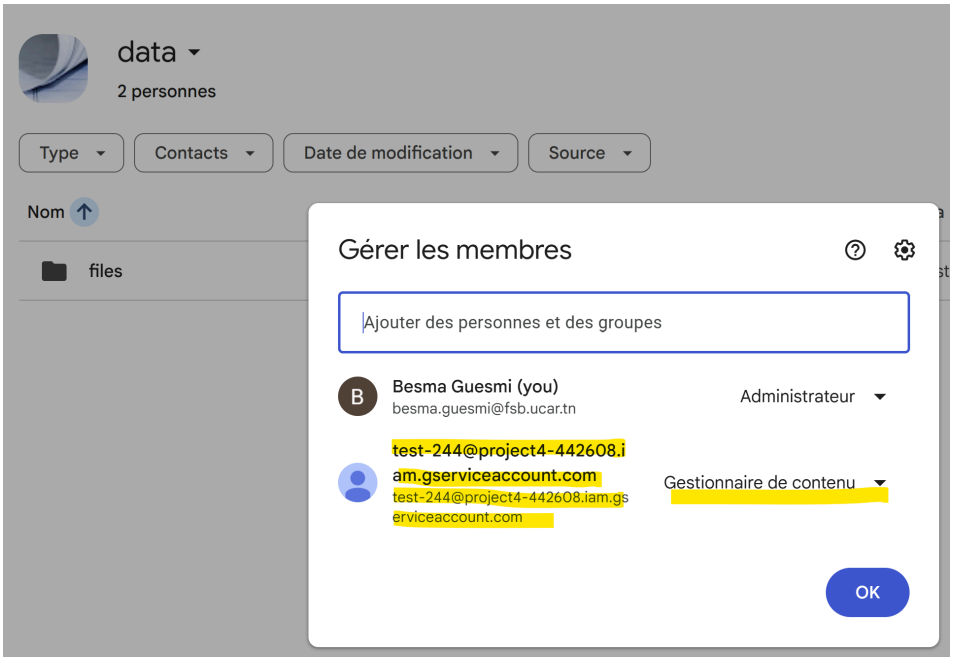
2.2 Google Drive

9. Then move to your Google Drive and create a folder where the data will be pushed (Saved), it should be a shared drive



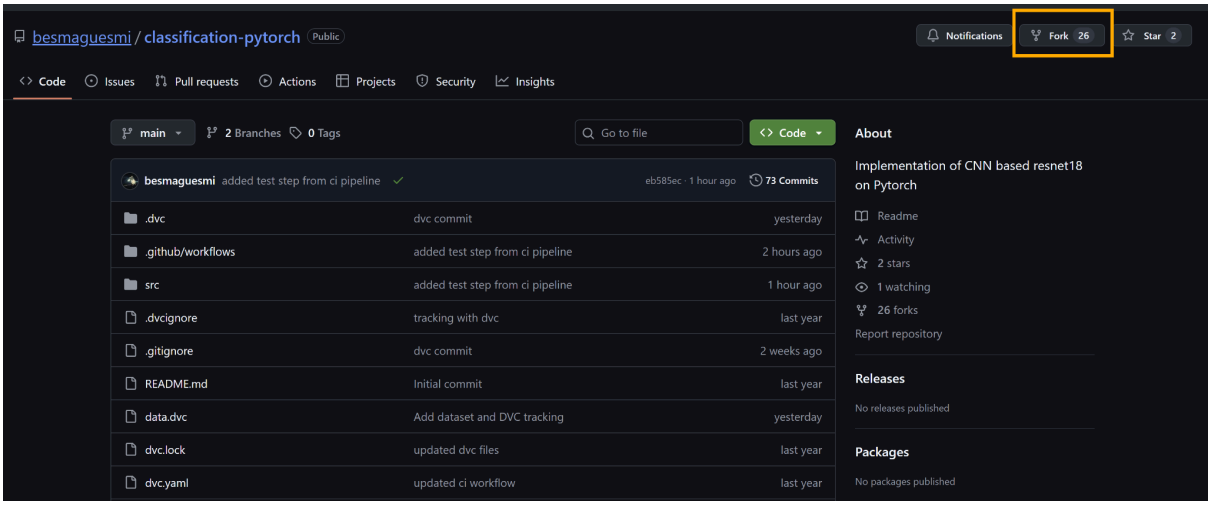
10. Please note that you should grant your shared drive folder permission to your service account and provide Editor access (step 5 above). You should save the folder ID since it would be needed for the DVC configuration. (e.g in this case: Folder ID is 1r3UU3LjsqjEpAubsCHfeMajgKoaOGbec)





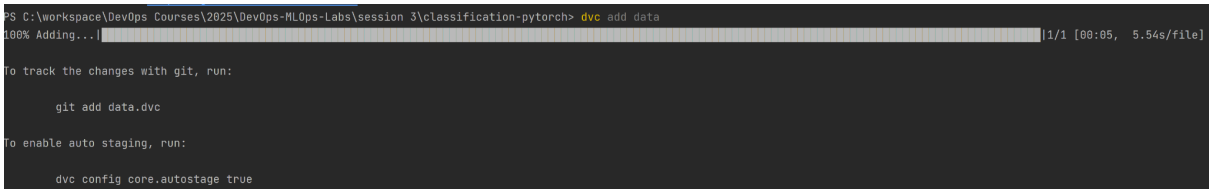
2.3 Local Setup

Please fork this repository first: https://github.com/besmaguesmi/classification-pytorch



11. Go to your project repository, and install the dependencies need including dvc: `$pip install dvc dvc-gdrive`
12. Then run `$dvc init` To initialise a new dvc project in your current repository.
13. Then, add your data by running `$dvc add data` (here my data folder is called "data") *(Please note that if Git already tracks your data folder, you should stop tracking it by git by adding it to.gitignore and add dvc tracking file instead)*

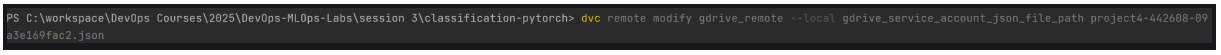
```
$git rm -r --cached 'data'
$git commit -m "stop tracking data"
$git add data.dvc
$git commit -m "Add dataset and DVC tracking"
```



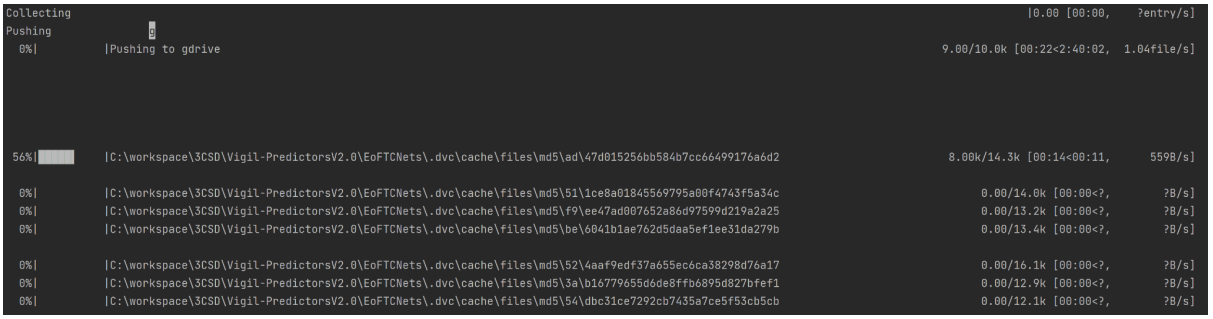
14. Then, add a Google Drive remote as the default storage location for DVC-tracked files, by running `dvc remote add -d gdrive_remote gdrive://FolderID (e.g dvc remote add -d gdrive_remote gdrive//:13CyP0NFEnH3-fIAckNi3Vj3JQI8Yz3bR)`

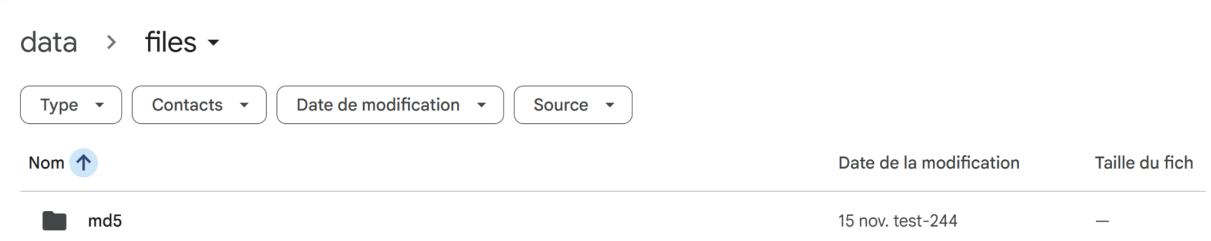
```
Setting 'gdrive_remote' as a default remote.
```

15. Then, configure DVC to use a Google Drive service account for authentication., by running: `dvc remote modify gdrive_remote gdrive_use_service_account true`
16. Enabling the acknowledgment of the abuse warning from Google Drive when using a service account, by running `dvc remote modify gdrive_remote gdrive_acknowledge_abuse true`
17. Specifies the path to the service account JSON file for authentication with Google Drive (which you already downloaded and saved under your repository): `dvc remote modify gdrive_remote --local gdrive_service_account_json_file_path data-management-441922-74cd3f254b92.json`



18. Enables automatic staging of DVC-tracked files for Git commits (Optional) by running `dvc config core.autostage true`. After that, you should have config and config.local files under .dvc folder which respectively stores global settings for the entire DVC repository (could be accessible for collaborators) and stores local, machine-specific settings, not shared or versioned (often used for credentials or personal paths).
19. You should also push any large files such as trained models (please download model from here) ignore them from git, and push them to the drive using `dvc add models/` (dvc remove train_model if needed) and then `git rm -r --cached 'models'` and then `git commit -m "stop tracking models"`
20. Finally, upload DVC-tracked files to the configured remote storage (Google Drive) using `dvc push` (PS: If you're pushing around 1GB of data, allow approximately 10 minutes for the process to complete)



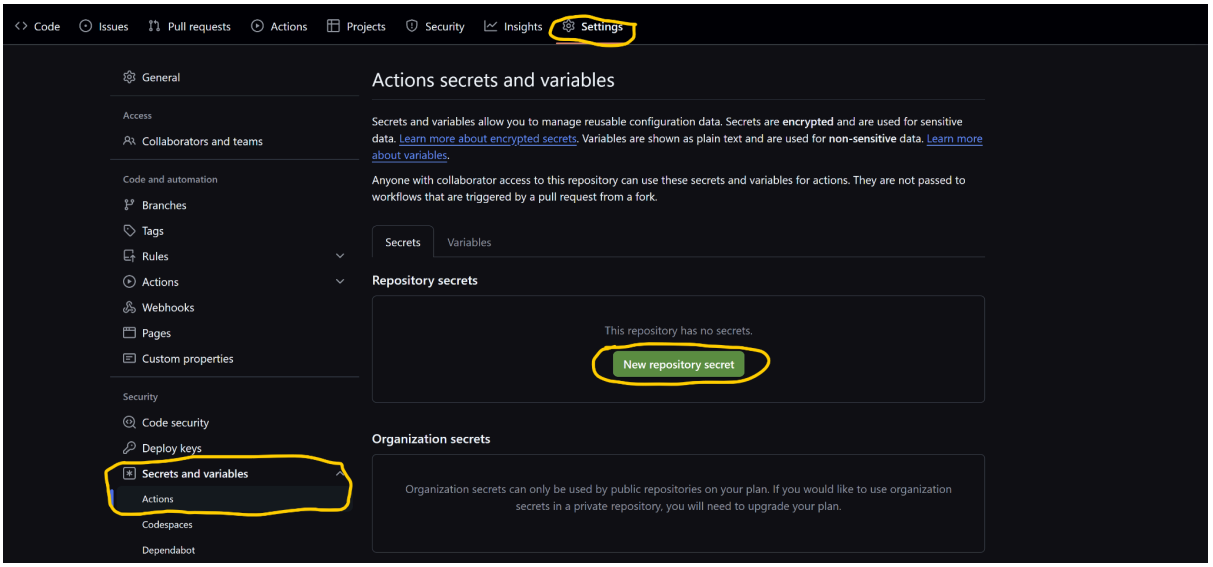


2.4 GitHub Actions

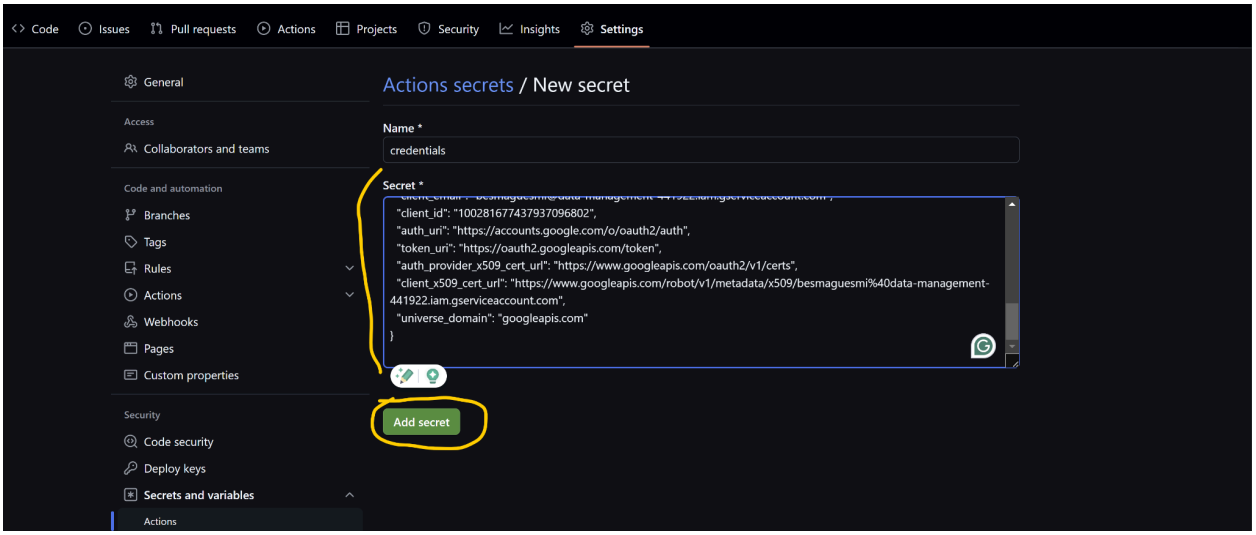
Push the changes to your GitHub repository. Please note that, by default, only the config and .gitignore files within the .dvc folder are tracked and pushed.

```
git add .dvc/config, .gitignore data.dvc
git commit -m “dvc commit”
git push
```

Once you push these changes to the repository, go to GitHub and navigate to your repository's settings tab. Under the "Secrets and variables" section, select "Actions" and then click on "Repository secrets."

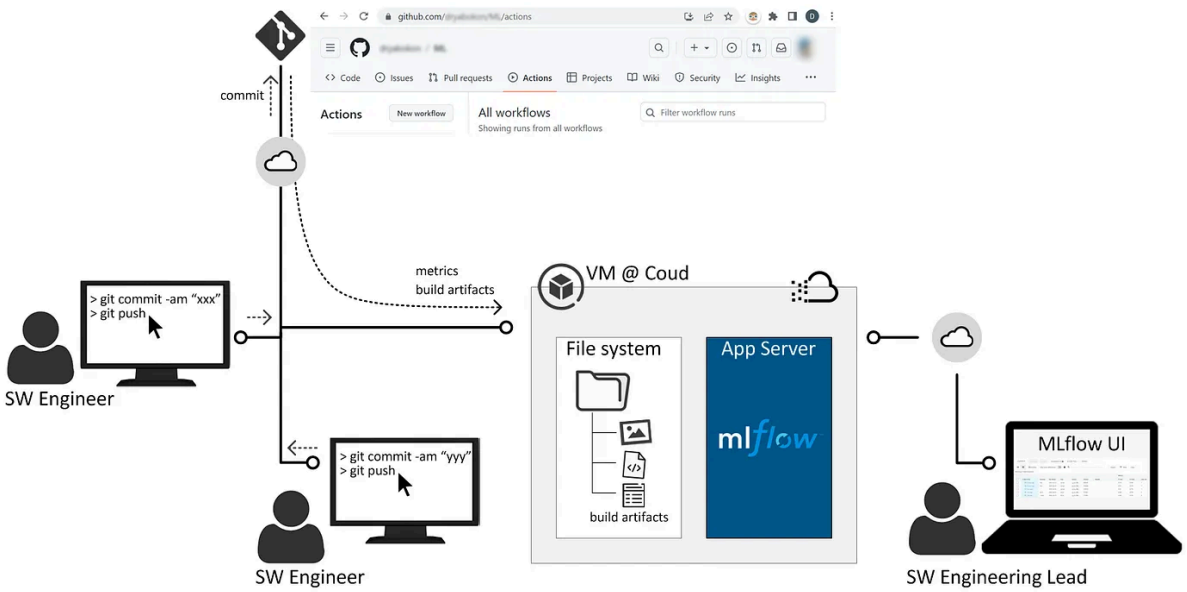


Then, create a new secret named GDRIVE_CREDENTIALS_DATA and paste the contents of your service account credentials JSON file into the value field.



2.5 MLflow Setup

MLflow is an open-source platform for managing the end-to-end machine learning lifecycle. This guide explains how to set up and use MLflow with our PyTorch classification project.



Install MLflow

```
# Using pip
pip install mlflow

# Using uv (recommended)
uv pip install mlflow

# For additional features (optional)
uv pip install mlflow[pytorch] boto3 psycopg2-binary
```

Step 1: Start MLflow Tracking Server

```
# Terminal 1 - Start MLflow UI
mlflow ui --host 0.0.0.0 --port 5000
```

Step 2: Access MLflow UI

Open your browser and navigate to: http://localhost:5000

Step 3: Training with MLflow

Please pull the latest updates using the command below:

git remote add upstream https://github.com/besmaguesmi/classification-pytorch.git

git fetch upstream

git checkout main

git merge upstream/main

And then, launch the training with MLflow:

python main.py --mode train --data_path data/train --use_mlflow

During Training, the MLflow tracks:

- ➢ Parameters: Learning rate, batch size, backbone architecture, epochs
- ➢ Metrics: Training/validation loss and accuracy (per epoch)
- ➢ Artifacts: Saved models, loss curves, training history
- ➢ Models: Versioned PyTorch models with metadata

Step 4: Testing with MLflow

python main.py --mode test --data_path data/test --model_path models/your_model.pth --use_mlflow

During Testing, MLflow tracks:

- ➢ Parameters: Model path, test dataset info
- ➢ Metrics: Test accuracy, precision, recall, F1-score
- ➢ Artifacts: Confusion matrices, classification reports
- ➢ Model Lineage: Link between training and testing runs